

turknet

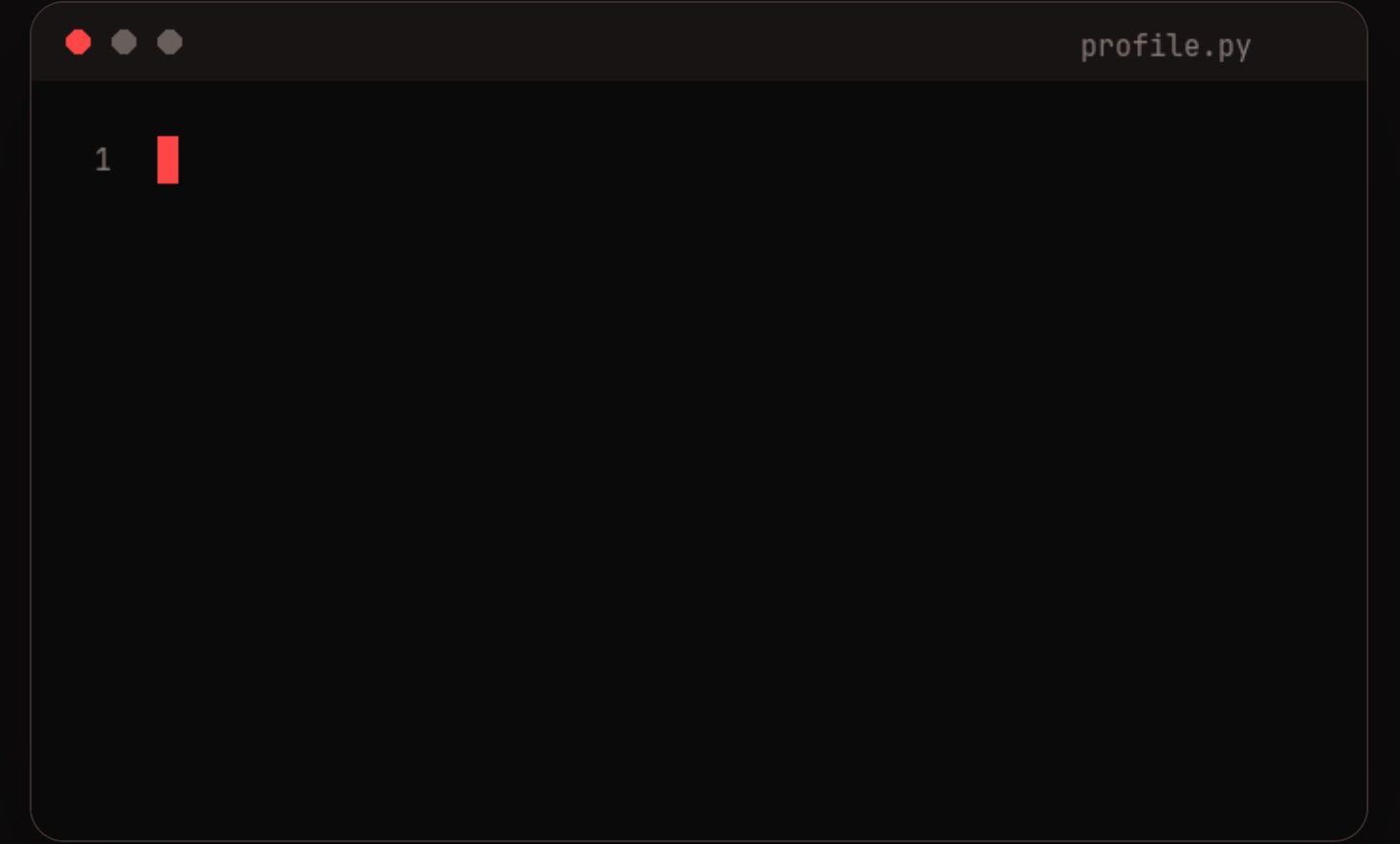
2026-27 GÜZ DÖNEMİ • HAFTA 2

Güven sınırını geçmek

Tehdit modelleme ve enjeksiyon: girdinin koda
dönüştüğü an

Can Öztaş

github.com/canoztas/secure-coding



BUGÜN

Bugünün akışı

01	Kısa tekrar	Geçen haftanın üç fikri: güvenilmeyen girdi, güven sınırı, saldırganın sorusu.
02	Tehdit modelleme	Bir sistemi saldırganın gözünden çizmek: DFD ve STRIDE.
03	Enjeksiyonun kök nedeni	Veri ile kodun aynı kanalda birleşmesi.
04	SQL, komut, yol	Aynı hatanın üç yüzü ve tek yapısal savunma.

TEKRAR

Geçen haftadan üç fikir

01

Güvenilmeyen girdi

Güven sınırını geçen her şey girdidir ve kanıtlanana kadar güvenilmezdir.

02

Güven sınırı

Kararlar sunucuda alınır; istemcide yapılan doğrulama isteğe bağlıdır.

03

İki soru

“Ne yapmalı?” testlerin sorusu. “Başka ne yapabilir?” saldırganın sorusu.

Bu hafta üçünü tek bir zafiyet sınıfında —enjeksiyon— birlikte göreceğiz.

BÖLÜM A

Tehdit modelleme

Kod yazmadan önce: bu sistemi kim, nereden, nasıl kırabilir?



YÖNTEM

Dört soru

SORU 1

Ne inşa ediyoruz?

Sistemi veri akış diyagramıyla (DFD) çiz: süreçler, depolar, aktörler, güven sınırları.

SORU 2

Ne ters gidebilir?

Her sınırı geçen okta STRIDE'ı uygula: hangi tehdit sınıfı mümkün?

SORU 3

Ne yapacağız?

Her tehdide bir karşı önlem; kabul edilen riski açıkça yaz.

SORU 4

İyi iş çıkardık mı?

Modeli kod ve testlerle karşılaştır; sistem değişince modeli güncelle.

Adam Shostack'ın dört sorusu. Model bir kez değil, sistem değiştikçe yenilenir.

Altı tehdit sınıfı

Harf	Tehdit	Kırdığı ilke	Örnek
S	Spoofing (kimlik taklidi)	Kimlik doğrulama	Başkasının oturumuyla giriş
T	Tampering (kurcalama)	Bütünlük	Fiyatı ya da rolü değiştirmek
R	Repudiation (inkâr)	İzlenebilirlik	İşlemi yapmadım demek, log yok
I	Information disclosure	Gizlilik	Veri sızıntısı
D	Denial of service	Erişilebilirlik	Servisi durdurmak
E	Elevation of privilege	Yetkilendirme	Kullanıcıdan yöneticiye çıkmak

Geçen haftanın kodunu modelleyelim

```
app.py
1  # trust boundary: browser  →|→  server
2  GET /profile?id=1
3
4  # process: profile()
5  # store:  users table
6  # flow:   id  →  SQL query  →  DB
```

Tampering + Information Disclosure

id alanı bir güven sınırını geçiyor ve doğrudan sorguya giriyor. STRIDE'in T ve I harfleri tam da burada. Kök neden hep aynı: veri, koda dönüşüyor.

Aynı diyagram komut satırında, dosya yollarında ve şablon motorlarında da tekrar eder.

BÖLÜM B

Enjeksiyon

OWASP Top 10 · A05:2025 — hâlâ en yaygın ve en ölümcül sınıflardan biri.



Tek bir kök neden: veri ile kod aynı kanalda.

Uygulama bir metin kuruyor; metnin bir kısmı sizin kodunuz, bir kısmı kullanıcının verisi. Yorumlayıcı ikisini ayırt edemez. Savunma kaçış karakteri eklemek değil, kanalları ayırmaktır.



Aynı hata, farklı yorumlayıcı

SQL

Veritabanı

Sorgu metnine giren girdi — CWE-89.

OS

Kabuk / komut

Komut satırına giren girdi — CWE-78.

PATH

Dosya sistemi

Yola giren girdi (../) — CWE-22.

TEMPLATE

Şablon motoru

İfade olarak yorumlanan girdi —
CWE-1336.

NoSQL

Belge sorgusu

Operatör enjekte eden girdi — CWE-943.

XML

Ayrıştırıcı

Harici varlık (XXE) — CWE-611.

SQL

Girdi sorguya dönüşünce



app.py

```
1 uid = request.args.get("id")           # untrusted
2 q = "SELECT name FROM users "
3 q += "WHERE id = " + uid                # data glued into query text
4 db.execute(q)
```

Beklenen

id = 1 → tek kullanıcı döner.

Girdi değişince

id, sorgunun mantığını değiştirecek şekilde seçilirse tek satır yerine tablonun tamamı dönebilir.

Enjeksiyon nasıl görünür

Tür	Fikir	Belirti
In-band	Sonuç doğrudan yanıtta görünür (UNION, hata mesajı).	Sayfada fazladan veri
Blind (boolean)	Yanıt doğru/yanlış'a göre değişir.	İçerik var/yok
Blind (zaman)	Koşullu gecikme ile bit bit okuma.	Yavaş yanıt
Second-order	Girdi saklanır, sonra başka sorguda tetiklenir.	Gecikmeli etki

BÖLÜM B1

Adım adım bir SQLi

Bir giriş formunu satır satır kıralım, sonra aynı formu düzeltelim.



Kırılgan giriş uç noktası

```
app.py

1 @app.route("/login", methods=["POST"])
2 def login():
3     name = request.form["name"]          # untrusted
4     pw = request.form["password"]        # untrusted
5     q = ("SELECT * FROM users WHERE "
6         "name = '" + name + "' AND password = '" + pw + "'")
7     return "ok" if db.execute(q).fetchone() else "401"
```

İki alan da doğrudan sorgu metnine yapışıyor. Şimdi üç istekle ne olduğunu izleyelim.

Beklenen davranış



query.sql

```
1 POST /login      name=alice      password=1234
2
3 # built query:
4 SELECT * FROM users WHERE name = 'alice' AND password = '1234'
```

Ne gitti

Kullanıcı adı ve parola beklenen biçimde.

Ne oldu

Eşleşen tek satır döner → giriş başarılı. Test bu senaryoyu görür ve geçer.

Parola hiç kontrol edilmedi



query.sql

```
1 POST /login      name=' OR '1'='1' --      password=x
2
3 # input closed the quote, -- commented out the rest:
4 SELECT * FROM users WHERE name = '' OR '1'='1' --' AND password = 'x'
```

Ne yaptı

'1'='1' her zaman doğru; parola koşulu -- ile silindi.

Ne çıktı

İlk satır (çoğu kez yönetici) döner → parolasız giriş.

UNION ile başka tabloyu ekle



query.sql

```
1 GET /profile?id=0 UNION SELECT name, password FROM users --
2
3 # built query:
4 SELECT name, email FROM users WHERE id = 0
5     UNION SELECT name, password FROM users --
```

Ne yaptı

id=0 hiç satır getirmez; UNION ikinci sorgunun sonucunu yanıtı ekler.

Ne çıktı

Tüm kullanıcıların adı ve parola özeti sayfada listelenir.

Dil değişti, kök neden aynı



Login.java

```
1 // Statement + string concatenation = same hole
2 String q = "SELECT * FROM users WHERE name = '" + name + "'";
3 Statement st = conn.createStatement();
4 ResultSet rs = st.executeQuery(q);
5 // name = ' OR '1'='1 → same as Step 2
```

SQLi bir dile ya da çerçeveye özgü değil; veriyle kodu aynı kanalda birleştiren her yerde çıkar.

Girdi artık veri, kod değil



```
1  # Python – parameterized
2  q = "SELECT * FROM users WHERE name = ? AND password = ?"
3  db.execute(q, (name, pw))
4
5  // Java – PreparedStatement
6  var ps = c.prepareStatement("... WHERE name = ?");
7  ps.setString(1, name);
```

Adım 2'yi tekrar dene

name = "' OR '1'='1' --" artık bu metinde bir kullanıcı adı aranır.

Sonuç

Böyle bir ad yok → eşleşme yok → 401. Sınıfın tamamı kapandı.

Yanıt görünmese de sızar



query.sql

```
1  -- 1) Boolean: response changes with true/false
2  id=1 AND (SELECT SUBSTR(password,1,1) FROM users LIMIT 1)='a'
3  --    page normal → char correct ;    empty → wrong
4
5  -- 2) Time-based: response is delayed if the condition is true
6  id=1 AND IF(SUBSTR(password,1,1)='a', SLEEP(3), 0)
```

Yanıta hiç veri görünmese bile bilgi bit bit çıkarılabilir; “çıktı yok” güvenlik değildir.

ORM tek başına korumaz



models.py

```
1 # Django – string formatting in raw SQL is still open
2 User.objects.raw("... WHERE name = '%s'" % name)    # BAD
3 User.objects.raw("... WHERE name = %s", [name])      # OK
4
5 # SQLAlchemy – same rule inside text()
6 session.execute(text("... WHERE name = '" + name + "'")) # BAD
7 session.execute(text("... WHERE name = :name"), {"name": name}) # OK
```

ORM'in normal sorgu kurucusu güvenlidir; tehlike `.raw()/.extra()/text()` ile metne döndüğünüz yerededir.

Savunma: hazır ifade (prepared statement).

Sorgunun iskeleti sabit derlenir; kullanıcı verisi ayrı bir kanaldan parametre olarak gider. Yorumlayıcı artık veriyi kod olarak okuyamaz — sınıfın tamamı kapanır.

SQL

Kanalları ayırmak



app.py

```
1 # BAD  data mixes into code
2 q = "... WHERE id = " + uid
3
4 # OK  fixed skeleton, data as a parameter
5 cur.execute(
6     "... WHERE id = %s", (uid,))
```

Aynı ilke her dilde var: sorgu metnine değil, ayrı parametre kanalına yaz.

Her dilde aynı ilke

```
1 // Java – PreparedStatement
2 var ps = c.prepareStatement("... WHERE id = ?");
3 ps.setInt(1, id);
4
5 // C# – parameterized command
6 cmd.CommandText = "... WHERE id = @id";
7 cmd.Parameters.AddWithValue("@id", id);
8
9 // PHP (PDO) and Node (pg)
10 $pdo->prepare("... WHERE id = ?")->execute([$id]);
11 client.query("... WHERE id = $1", [id]);
```

Yer tutucu farklı (?, @id, \$1) ama fikir tek: veri asla sorgu metni olmaz.

Parametrelenemeyeni izin listesiyle bağla

```
1 # BAD  sort column comes straight from input
2 q = f"SELECT * FROM product ORDER BY {col}"  # col = ?
3
4 # OK  column/table name can't be a parameter → allowlist
5 ALLOWED = {"name", "price", "date"}
6 if col not in ALLOWED:
7     raise ValueError('invalid column')
8 q = f"SELECT * FROM product ORDER BY {col}"  # now safe
```

Tablo adları, kolon adları, ORDER BY yönü parametrelenemez; hepsini sabit bir kümeye eşle.

BÖLÜM B2

Diğer yorumlayıcılar

Aynı adımları komut, dosya yolu, NoSQL ve şablon için de izleyelim.



Girdi kabuk komutuna dönüşünce



app.py

```
1 host = request.args["host"]      # untrusted
2 os.system("ping -c1 " + host)    # shell string is built
3
4 # input:  host = 8.8.8.8 ; id
5 # runs:   ping -c1 8.8.8.8 ; id  ← second command runs too
```

Ne gitti

host alanına ; ile ikinci bir komut eklendi.

Ne oldu

Kabuk ; işaretini komut ayırıcı okur;
ikinci komut çalışır.

Düzeltilme

subprocess.run(..., shell=False): kabuk
yok, girdi argüman kalır.

KOMUT · ARGÜMAN

Kanal ayrıldı, risk bitmedi



app.py

```
1 # no shell, but input still becomes an ARGUMENT
2 subprocess.run(["git", "log", user_input])
3 #   input could be "--output=/etc/cron.d/x" → option injection
4
5 # OK   separate values with -- and validate the format
6 subprocess.run(["git", "log", "--", path])
```

Argüman enjeksiyonu (CWE-88): girdi komut değil ama bir bayrak gibi yorumlanır. -- ayırıcı ve allowlist şart.

Girdi dosya yoluna dönüşünce



app.py

```
1 path = request.args["file"]          # untrusted
2 open("/data/" + path)                 # glued into the path
3
4 # input:  file = ../../etc/passwd
5 # opened: /data/../../etc/passwd → /etc/passwd
```

Ne gitti

Dosya yoluna ../ ile üst dizine çıkış eklendi.

Ne oldu

Yol /data kökünün dışına taştı; sistem dosyası okundu.

Düzeltilme

realpath ile kanonikleştir, sonra kökün içinde mi diye doğrula.

Girdi tipi değişince koşul çöker



app.js

```
1 // expected: match two strings
2 db.accounts.find({ username: userIn, password: passIn })
3
4 // input:  password = { "$ne": null } (an object, not a string)
5 // query:  password ≠ null → always true → bypass
```

Ne gitti

parola alanına JSON gövdesinde bir operatör nesnesi geldi.

Ne oldu

Eşitlik yerine \$ne operatörü çalıştı; koşul her zaman doğru.

Düzeltilme

if (typeof passIn !== "string") reject():
tipi doğrula.

Girdi şablonun kendisi olunca



app.py

```
1  render_template_string("Hello " + name)    # data became the TEMPLATE
2
3  # input:    name = {{ 7*7 }}
4  # screen:  Hello 49                        ← the engine executed the input
5
6  # OK  data goes into the template as a PARAMETER
7  render_template("hello.html", name=name)  # {{ name }} is text only
```

Ne oldu

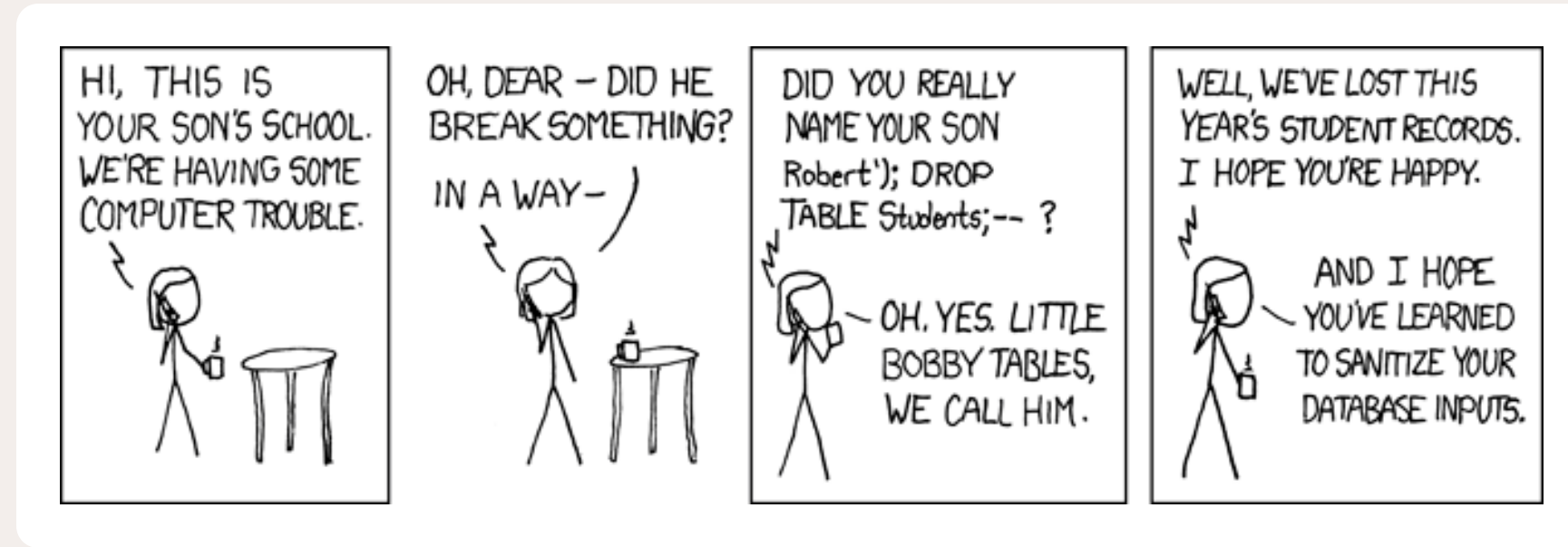
7*7 = 49 çıktıysa motor girdiyi kod olarak işledi; buradan RCE'ye gider (CWE-1336).

Akraba: XXE

XML'de de aynı sınıf: harici varlıkları ve DTD'yi kapat; ayrıştırıcı varsayılanı güvensiz (CWE-611).

KLASİK

xkcd 327 — “Little Bobby Tables”



Okul öğrencinin adını doğrudan sorguya koymuş. Espri gerçek: girdi her zaman veri kalmalı.

BÖLÜM C

Vaka otopsisı

MOVEit Transfer · CVE-2023-34362 — tek bir SQLi'nin nereye kadar gittiği.



MOVEit Transfer • CVE-2023-34362

Ürün

Kurumların dosya taşıdığı yönetilen transfer yazılımı (MOVEit Transfer).

Kök neden

web uygulamasının /human.aspx istek işleyicisi, kullanıcı girdisini SQL'e sanitize etmeden koyuyor — CWE-89.

Mayıs 2023, sıfıncı gün (zero-day). Cl0p (TA505) kitlesel sömürdü; yüzlerce kurumdan veri sızdı.

Bir enjeksiyondan tam ele geçirmeye

- | | | |
|----|------------------------------|---|
| 01 | SQL enjeksiyonu | /human.aspx'e hazırlanmış istek → veritabanı şeması ve kimlik bilgileri okunur. |
| 02 | Ayrıcalık | DB'den yönetici (sysadmin) API erişim token'ı çekilir / sahte oturum kurulur. |
| 03 | Deserialization → RCE | Token'la API'deki güvensiz nesne çözme noktasına payload → kod çalıştırma. |
| 04 | Kalıcılık | human2.aspx (LEMURLOOT) web shell wwwroot'a bırakılır → toplu veri sızdırma. |

Zincirin ilk halkası tek bir SQLi. O halka kırılıyorsa geriye çökerdi.

Hangi desen açtı? (temsîlî)

```
1 // SQL built with string concatenation in the request handler
2 string q = "SELECT ... WHERE x = '" + Request["p"] + "'";
3 db.Exec(q); // BAD input = code
4
5 // what it should be: a parameterized command
6 cmd.CommandText = "SELECT ... WHERE x = @p";
7 cmd.Parameters.AddWithValue("@p", Request["p"]);
```

Tek bir uç noktadaki birleştirme bütün zinciri açtı. “Çalışan kod, güvenli kod değildir.”

Otopsi raporunun dört başlığı

KOD

Kanalı ayır

Her uç noktada parametrelili sorgu / ORM binding. String birleştirme kalmasın.

YAMA

Sürümü yükselt

Progress yamaları: 2021.0.6 / 2021.1.4 / 2022.0.4 / 2022.1.5 / 2023.0.1.

AZALT

Saldırı yüzeyi

Dışa açık HTTP/S'i kıs, human2.aspx ve IOC'leri ara, DB'ye en az yetki.

ÇIKARIM

Ders

Tek zafiyet = tam ihlal. Girdi doğrulama + savunmayı katmanla.

Dönem projeniz (bireysel otopsi) tam bu dört başlığı içermeli: kök neden, zincir, yama, çıkarım.

ÖZET

Bir cümlede

KÖK NEDEN

Aynı kanal

Veri ile kod tek metinde birleşiyor.

SAVUNMA

Kanalı ayır

Parametre, argüman dizisi, kanonik yol, güvenli şablon.

TASARIM

En az yetki

Kırılsa bile hasar dar kalsın.

HAFTAYA

Haftaya kadar

PortSwigger

Web Security Academy'de SQL injection ve OS command injection laboratuvarlarından birer tanesini bitirin.

Kendi ortamınız

DVWA ya da Juice Shop'u Docker'da çalıştırın; istekleri Burp'ten izleyin.

Haftaya: tarayıcı güvenlik modeli, XSS ve CSRF. Etik: yalnızca kendi laboratuvarınızda.



SORU • CEVAP

Sorular?

